

Federated Variational Preference Alignment with Gumbel-Softmax Prior for Personalized User Preferences

Anonymous Authors¹

Abstract

Federated Learning (FL) offers a privacy-preserving pathway for aligning Large Language Models (LLMs); however, existing frameworks typically enforce a monolithic reward model, inevitably averaging out inherently conflicting user preferences (e.g., helpfulness vs. harmlessness). While Variational Preference Learning (VPL) offers a pathway to personalization, adapting it to decentralized settings presents a fundamental challenge: *posterior collapse* driven by severe local data scarcity and heterogeneity. In this paper, we propose Federated Variational Preference Alignment with Gumbel-Softmax Prior (FedVPA-GP), a framework designed to disentangle diverse preferences without compromising privacy. To stabilize variational inference, we introduce a *Federated Mixture Prior* that enables clients to leverage the aggregate population distribution as a dynamic prior. Furthermore, we incorporate an *Orthogonal Loss* that explicitly enforces the separation of preference prototypes in the latent space. Experiments on the HH-RLHF dataset demonstrate that FedVPA-GP significantly outperforms monolithic baselines, successfully disentangling conflicting user intents and enabling dynamic preference switching.

1. Introduction

Reinforcement Learning from Human Feedback (RLHF) has established itself as the standard paradigm for aligning Large Language Models (LLMs) with human intent (Christiano et al., 2017; Ziegler et al., 2019; Ouyang et al., 2022). However, its reliance on centralized data aggregation poses a critical bottleneck: high-quality preference data—often

reflecting personal, cultural, and political nuances—resides on edge devices. Centralizing this data not only risks severe privacy violations, such as the extraction of sensitive training data (Carlini et al., 2021), but also faces challenges with regulations like GDPR (European Parliament and Council of the European Union, 2016).

Federated Learning (FL) offers a privacy-preserving alternative (McMahan et al., 2017; Kairouz et al., 2021). Recent frameworks have adapted alignment to this decentralized setting, such as *FedDPO* (Ye et al., 2024) and *FedBiscuit* (Wu et al., 2024). Notably, *FedBiscuit* addresses computational constraints by training only a binary preference selector on the client side while keeping the base LLM frozen (Wu et al., 2024). However, despite these advancements, existing frameworks share a critical limitation: they enforce a monolithic reward model. Human values are inherently pluralistic and can be conflicting—for instance, preferences often diverge between helpfulness and harmlessness (Santurkar et al., 2023; Poddar et al., 2024). Aggregating these heterogeneous distributions into a single global model yields theoretical sub-optimality (Shirali et al., 2025). By aiming for a monolithic solution, these methods implicitly enforce a consensus that does not exist, resulting in a one-size-fits-all model that fails to satisfy distinct client needs.

While *Variational Preference Learning* (VPL) (Poddar et al., 2024) offers a pathway to personalization by modeling user intent as a latent variable, adapting it to the federated setting presents a fundamental challenge driven by two intrinsic characteristics of FL: data heterogeneity and data scarcity. In centralized regimes, the model learns a dense preference manifold from pooled data, allowing it to distinguish subtle variations in user intent. In contrast, federated clients operate on highly heterogeneous distributions, where each client observes only a fragmented slice of global preferences, often restricted to a single mode like helpfulness or harmlessness. Compounding this, the severe scarcity of local samples causes the KL regularization term to dominate the reconstruction objective during variational inference. Lacking both the global context to position their preferences and sufficient data to support complex posterior estimation, the latent variable often degenerates to an uninformative prior. This phenomenon, known as **posterior collapse**, renders the personalization mechanism ineffective in decentralized

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Federated Variational Preference Alignment with Gumbel-Softmax Prior (FedVPA-GP)

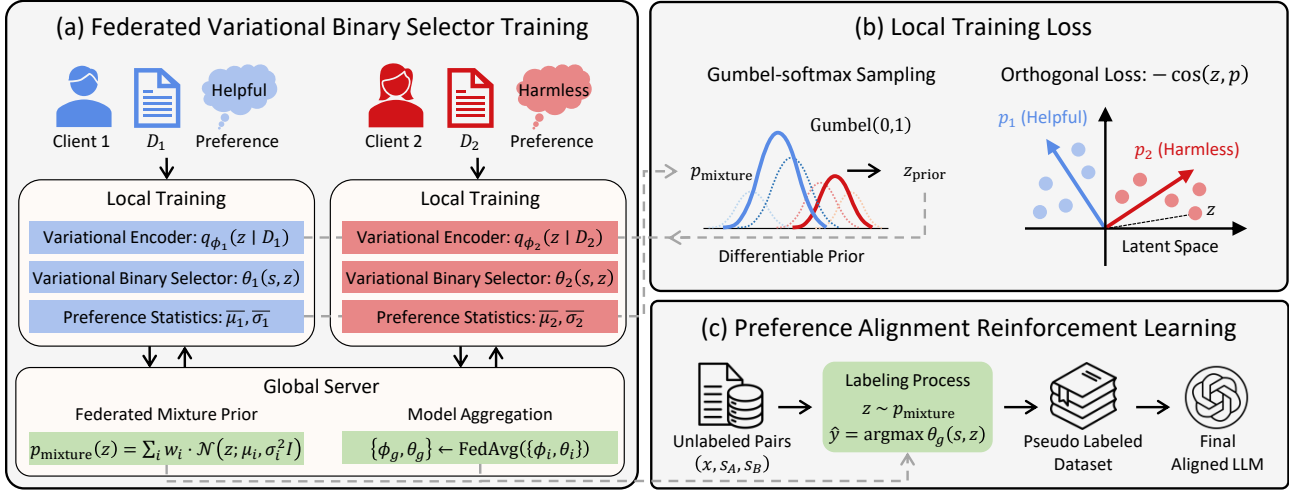


Figure 1. Overview of the proposed FedVPA-GP framework. (a) Illustrates the federated training process of the variational binary selector. (b) Details the local variational objective designed to enhance inference. (c) Depicts the subsequent preference alignment stage using the trained selector.

environments (Bowman et al., 2016; Alemi et al., 2018).

To overcome these challenges, we propose Federated Variational Preference Alignment with Gumbel-Softmax Prior (FedVPA-GP). We bridge the gap between local data sparsity and global distribution requirements through two core mechanisms. First, we introduce a **Federated Mixture Prior** that aggregates learned distributions from other clients, serving as a dynamic prior that stabilizes local inference. Second, to explicitly prevent posterior collapse and ensure semantic disentanglement, we incorporate an **Orthogonal Loss** that enforces the separation of conflicting preference prototypes in the latent space. By combining these with Gumbel-Softmax relaxation for end-to-end differentiability (Jang et al., 2017), FedVPA-GP successfully learns personalized reward models without sharing raw data as shown in the Figure 2b.

Extensive experiments on the HH-RLHF dataset (Bai et al., 2022) demonstrate that FedVPA-GP significantly outperforms monolithic baselines. Qualitative analysis further confirms that our algorithm successfully disentangles preferences in the latent space, enabling the model to dynamically switch between helpful and harmless modes based on the inferred context.

Our contributions are summarized as follows:

- **Federated Variational Preference Alignment:** We address the limitation of monolithic reward models in capturing conflicting user preferences. By integrating variational inference into Federated Learning, our framework effectively adapts to diverse user intents while preserving data privacy.

- **Stabilized Variational Inference and Disentanglement:** To overcome data scarcity and prevent posterior collapse in federated settings, we introduce a mechanism combining a *Federated Mixture Prior* with an *Orthogonal Loss*. This approach stabilizes posterior estimation and enforces the semantic separation of distinct preference prototypes.
- **Empirical Validation:** Experiments on the HH-RLHF dataset (Bai et al., 2022) demonstrate that FedVPA-GP significantly outperforms monolithic baselines (e.g., FedBiscuit, FedDPO) with robust generalization to unseen clients. Qualitative analysis confirms that our model successfully disentangles preferences.

2. Related Works

Reinforcement Learning from Human Feedback (RLHF) Since the seminal work of Christiano et al. (2017), RLHF has become a standard framework for aligning LLMs. The typical pipeline involves training a reward model on preference pairs to guide policy optimization via PPO (Schulman et al., 2017; Ouyang et al., 2022). Recently, methods such as Direct Preference Optimization (DPO) (Rafailov et al., 2023), IPO (Azar et al., 2024), and KTO (Ethayarajh et al., 2024) have been proposed to stabilize training by optimizing the policy directly without an explicit reward model. These approaches primarily operate in centralized settings, assuming access to aggregated datasets. Applying them to scenarios where data is distributed across edge devices introduces challenges related to data privacy and regulatory compliance (European Parliament and Council of the European Union, 2016).

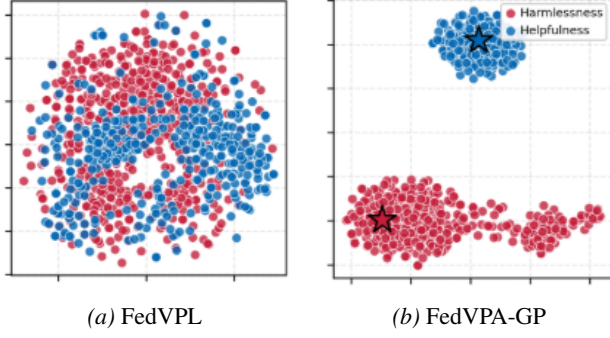


Figure 2. **Visualization of Latent Variable Distributions.** (a) FedVPL suffers from posterior collapse, where latent codes z_{VPL} cluster indistinguishably. (b) Our method (**FedVPA-GP**) effectively disentangles user preferences, showing distinct modes in $z_{\text{FedVPA-GP}}$ corresponding to different client groups.

Federated Preference Alignment To address privacy concerns, recent studies have integrated alignment techniques with Federated Learning. Wu et al. (2024) proposed *FedBiscuit*, which utilizes client-side adapters to learn preference representations. Similarly, Ye et al. (2024) introduced *FedDPO*, extending DPO to the federated setting by aggregating gradients to update a global policy. These frameworks generally aim to learn a global consensus model. While effective for privacy, this global aggregation approach tends to average the preference distributions across clients, which may limit the model’s flexibility in scenarios where user preferences are heterogeneous or conflicting (Shirali et al., 2025).

Personalized and Pluralistic Alignment Recognizing the diversity of human values (Santurkar et al., 2023), researchers have explored personalization in centralized settings. Techniques include multi-objective optimization (Rame et al., 2023), attribute steering (Dong et al., 2023), and weight merging (Jang et al., 2023). Notably, *Variational Preference Learning (VPL)* (Poddar et al., 2024) models user intent as a latent variable to capture continuous preference manifolds. However, these methods typically require access to the full dataset to learn the latent structure. Extending such variational approaches to Federated Learning presents specific challenges, particularly regarding local data sparsity and the estimation of stable posteriors in isolated environments.

3. Preliminaries

We consider a Federated Learning (FL) system consisting of K clients. Each client $i \in \{1, \dots, K\}$ has a private dataset of pairwise preferences $\mathcal{D}_i = \{(s_A, s_B, y)\}$, where s_A and s_B are two candidate responses from a Large Language Model (LLM), and $y \in \{0, 1\}$ indicates the user’s preference (with $y = 1$ denoting $s_A \succ s_B$).

3.1. Standard Federated Preference Alignment

In standard Federated RLHF settings, the goal is to learn a global reward model $r_\theta(s_A, s_B)$ that maximizes the likelihood of user preferences across all clients. The preference probability is typically modeled using the Bradley-Terry-Luce (BTL) model (Bradley & Terry, 1952; Luce, 1959):

$$p_\theta(y = 1 | s_A, s_B) = \sigma(r_\theta(s_A) - r_\theta(s_B)), \quad (1)$$

where $\sigma(\cdot)$ is the sigmoid function. The federated objective minimizes the aggregate negative log-likelihood: $\min_\theta \sum_{i=1}^K \mathbb{E}_{\mathcal{D}_i} [-\log p_\theta(y | s_A, s_B)]$.

However, this formulation assumes a single consensus reward function r_θ , which inevitably averages out conflicting preferences (e.g., “Helpful” vs. “Harmless”) and fails to capture user-specific nuances (Poddar et al., 2024).

3.2. Variational Preference Learning (VPL)

To address heterogeneity, we adopt a latent conditional framework. We assume each user i is governed by a continuous latent preference vector $z_i \in \mathbb{R}^d$ that conditions the reward model. For binary choice tasks, we condition the model’s logits on z_i through a learned projection network:

$$\text{logits}(s_A, s_B | z_i) = \text{logits}_{\text{base}}(s_A, s_B) + f_\theta(z_i), \quad (2)$$

where $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{|\mathcal{C}|}$ is a learned linear projection (latent projection) that maps the latent vector to logit adjustments, and $\mathcal{C}$ is the set of choices (typically $\{A, B\}$). The choice probability is then computed via softmax over the conditioned logits.

Since z_i is unobserved, we treat it as a latent variable and employ Variational Inference (VI) (Kingma & Welling, 2014). We introduce a local variational posterior $q_\phi(z | \mathcal{D}_i) = \mathcal{N}(z; \mu_i, \sigma_i^2 I)$ parameterized by ϕ to approximate the true posterior. The encoder extracts preference features (e.g., embedding difference $\Delta h = h_{\text{chosen}} - h_{\text{rejected}}$) and outputs posterior parameters (μ_i, σ_i^2) . The latent vector is sampled using the reparameterization trick: $z_i = \mu_i + \sigma_i \odot \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$ and $\sigma_i = \exp(0.5 \cdot \log \sigma_i^2)$; $\odot$ denotes element-wise multiplication. For brevity in the method (Sec. 4.1), we define

$$q_i := q_\phi(z | \mathcal{D}_i), \quad \mathcal{N}_0 := \mathcal{N}(0, I). \quad (3)$$

Thus q_i denotes client i ’s variational posterior, and $\mathcal{N}_0$ the standard Gaussian prior.

The objective is to maximize the Evidence Lower Bound (ELBO):

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z | \mathcal{D}_i)} [\log p_\theta(\mathcal{D}_i | z)] - \beta \mathbb{D}_{KL}(q_i \| p(z)), \quad (4)$$

where $p(z)$ is the prior over latent preferences and β is a regularization coefficient. In the baseline VPL ablation we use $p(z) = \mathcal{N}_0$.

3.3. Limitations of Federated Variational Preference Learning

Transposing VPL to FL introduces critical challenges stemming from *preference heterogeneity* and *data sparsity*, which monolithic priors fail to address.

Sparsity and Instability: Data fragmentation leaves each client with a small local dataset $|\mathcal{D}_i|$, causing the variational posterior q_i to be estimated with high variance. Insufficient samples lead to unstable gradients and poor convergence when training from scratch. To address this, we propose a Federated Mixture Prior, which leverages the aggregated distributions of other clients as a dynamic prior. This mechanism stabilizes local inference by transferring global knowledge, allowing clients to learn reliable posteriors even with sparse data.

Heterogeneity and Posterior Collapse: In centralized VPL, the model learns a global latent structure from pooled data. In FL, however, clients infer in isolation using a generic standard Gaussian prior $\mathcal{N}_0$. This lack of global guidance often leads to *posterior collapse*, where the latent variable z degenerates to the uninformative prior and fails to encode personalized preferences (Bowman et al., 2016; Alemi et al., 2018). As illustrated in Figure 2b, this results in an entangled latent space where distinct preference clusters fail to emerge. To prevent this collapse and enforce a semantically meaningful structure, we introduce an **Orthogonal Loss** (Sec. 4.2), which explicitly separates conflicting preference prototypes.

4. Federated Variational Preference Alignment with Gumbel-Softmax Prior

We propose Federated Variational Preference Alignment with Gumbel-Softmax Prior (FedVPA-GP), a framework designed to learn personalized reward models in a privacy-preserving manner. Unlike previous approaches that simply aggregate gradients (Wu et al., 2024; Ye et al., 2024), FedVPA-GP treats user personalization as a distributed continuous latent variable inference problem. We first detail our variational inference mechanism with the proposed mixture prior, followed by the orthogonal regularization for disentanglement and finally the two-stage training strategy.

4.1. Variational Inference with Federated Mixture Prior

Inference Network: Each client maintains a local variational encoder $q_\phi(z | \mathcal{D}_i)$. Given a preference pair (s_A, s_B, y) , we extract hidden representations h_A, h_B from the frozen base LLM. Crucially, to isolate the preference signal from generic semantics, we explicitly construct the difference embedding as a difference in hidden state $\Delta h = h_{\text{chosen}} - h_{\text{rejected}}$ and excluding prompt tokens.

The difference vector Δh is processed by a feature extractor (a multi-layer perceptron) that transforms the raw embedding difference into a lower-dimensional feature representation. This feature extractor learns to distill preference-specific signals while suppressing general response characteristics. The processed features are then passed to the variational encoder to parameterize the local posterior distribution $q_\phi(z | \mathcal{D}_i)$.

$$q_\phi(z | \mathcal{D}_i) = \mathcal{N}(z; \mu_i, \sigma_i^2 I). \quad (5)$$

We employ the reparameterization trick $z_i = \mu_i + \sigma_i \odot \epsilon$, with $\epsilon \sim \mathcal{N}(0, I)$ to enable gradient-based optimization. By conditioning on Δh , our design forces z_i to encode the relative direction of user preferences rather than static response content.

Federated Mixture Prior with Learnable Gumbel-Softmax Weights: To mitigate local data sparsity, we leverage the population-level distribution as a dynamic prior. However, simply averaging distributions from all clients is suboptimal due to preference heterogeneity. To address this, we propose a Federated Mixture Prior with learnable weights. Let $\mathcal{S} \subseteq \{1, \dots, K\}$ be the set of participating clients. We construct the mixture prior $p_{\text{mixture}}^{(i)}(z)$ as a weighted sum of peer posteriors $\mathcal{N}_j(z)$:

$$p_{\text{mixture}}^{(i)}(z) = \sum_{j \in \mathcal{S}} w_j \cdot \mathcal{N}_j(z), \quad (6)$$

where w_j represents the relevance weight of client j 's distribution to the current client i .

To compute the KL divergence $\mathbb{D}_{KL}(q_i \| p_{\text{mixture}}^{(i)})$ stably, we employ the log-sum-exp trick for the log-mixture probability:

$$\log p_{\text{mixture}}^{(i)}(z) = \max_j a_j + \log \left(\sum_{j \in \mathcal{S}} \exp(a_j - \max_k a_k) \right), \quad (7)$$

where $a_j = \log w_j + \log \mathcal{N}_j(z)$. This formulation prevents numerical underflow when aggregating probabilities from numerous peers.

Gumbel-Softmax Relaxation: Instead of static weighting, we optimize these coefficients to prioritize compatible peers using the Gumbel-Softmax relaxation (Jang et al., 2017). The weights are computed via the reparameterization trick:

$$w_j = \frac{\exp((\log \pi_j + g_j)/\tau)}{\sum_{k \in \mathcal{S}} \exp((\log \pi_k + g_k)/\tau)}, \quad (8)$$

where π_j are learnable logits, $g_j \sim \text{Gumbel}(0, 1)$ is Gumbel noise, and τ is the temperature. By minimizing the KL divergence, the model automatically learns to upweight informative peers with similar preference structures while filtering out conflicting noise.

Algorithm 1 Server-Side: Federated Aggregation, Prior Management, and Stage 2 RLHF

Require: Clients K , rounds T , KL weight β , orthogonal weight λ

Ensure: Global parameters θ^T, ϕ^T , client z distributions $\{\bar{\mu}_i^T, \bar{\sigma}_i^2\}_{i=1}^K$; fine-tuned policy (Stage 2)

- 1: **Stage 1: Federated selector training**
- 2: Initialize θ^0, ϕ^0 (base model frozen, only VPL components trainable)
- 3: **for** round $t = 1, 2, \dots, T$ **do**
- 4: Sample clients $\mathcal{S}^t \subseteq \{1, \dots, K\}$ (typically $|\mathcal{S}^t| = 10$)
- 5: Broadcast θ^t, ϕ^t to $\mathcal{S}^t$
- 6: **if** $t > 1$ **then**
- 7: Broadcast mixture prior $\{(\mu_j, \sigma_j^2), w_j\}_{j \in \mathcal{S}^{t-1}}$ to $\mathcal{S}^t$
- 8: **end if**
- 9: **Wait for client updates**
- 10: Receive $(\theta_i^t, \phi_i^t, \bar{\mu}_i, \bar{\sigma}_i^2, n_i)$ from each client $i \in \mathcal{S}^t$
- 11: Aggregate: $\theta^{t+1} \leftarrow \frac{1}{|\mathcal{S}^t|} \sum_{i \in \mathcal{S}^t} \theta_i^t$
- 12: Aggregate: $\phi^{t+1} \leftarrow \frac{1}{|\mathcal{S}^t|} \sum_{i \in \mathcal{S}^t} \phi_i^t$
- 13: Collect z -distributions: $\{(\bar{\mu}_i, \bar{\sigma}_i^2, n_i)\}_{i \in \mathcal{S}^t}$
- 14: Compute weights: $w_i \leftarrow n_i / \sum_{j \in \mathcal{S}^t} n_j$ for all $i \in \mathcal{S}^t$
- 15: Store mixture prior: $\{(\bar{\mu}_i, \bar{\sigma}_i^2), w_i\}_{i \in \mathcal{S}^t}$ for next round
- 16: **end for**
- 17: **Stage 2: Conditional RL (selector as reward)**
- 18: Load VPL components from selector: encoder q_ϕ , feature extractor, Z-TO-EMBEDDING
- 19: Load client average $z \{\bar{\mu}_i\}_{i=1}^K$ (or compute from selector + training data)
- 20: Freeze selector (θ^T, ϕ^T); use logits($s_A, s_B | z$) as reward
- 21: **for** each DPO step (on server, no federated rounds) **do**
- 22: Get z for batch: from data, or $\bar{\mu}_i$ by client i , or infer via $q_\phi(\cdot | \text{features})$
- 23: Inject z into policy: inputs_embeds $\leftarrow$ base_embeds + Z-TO-EMBEDDING(z)
- 24: Generate conditioned on z ; score with selector logits($s_A, s_B | z$); update policy via DPO
- 25: **end for**

4.2. Orthogonal Loss for Preference Separation

To ensure the latent space semantically separates diverse preference modes and prevents posterior collapse, we introduce an orthogonal loss motivated by (Li et al., 2024). We maintain a set of M learnable prototype vectors $\{\mathbf{p}_m\}_{m=1}^M \subset \mathbb{R}^d$.

Prototype Initialization: We initialize these prototypes using QR decomposition (Saxe et al., 2013). This process transforms a random initialization into a strictly orthonormal basis, ensuring that prototypes begin in mutually orthogonal subspaces. The resulting basis is then projected to a fixed radius to guarantee sufficient separation from the origin.

Server-Side Label Assignment: To guide this separation, the server performs balanced k -means clustering (with $k = M$) on the collected client means $\{\bar{\mu}_i\}$ and assigns a prototype index $y_i^* \in \{1, \dots, M\}$ to each client.

Loss Computation: Clients encourage their latent z to align with the assigned prototype $\mathbf{p}_{y_i^*}$ while maintaining

Algorithm 2 Client-Side: Local Variational Training

Require: Local dataset $\mathcal{D}_i$, global parameters θ^t, ϕ^t , mixture prior $p_{\text{mixture}}(z)$ (if $t > 1$), local steps E , learning rate η , KL weight β , orthogonal weight λ

Ensure: Updated parameters θ_i^t, ϕ_i^t , average z distribution $(\bar{\mu}_i, \bar{\sigma}_i^2)$, sample size n_i

- 1: Receive θ^t, ϕ^t from server
- 2: **if** mixture prior received **then**
- 3: Update local prior: $p_{\text{mixture}}(z) \leftarrow \{(\mu_j, \sigma_j^2), w_j\}_{j \in \mathcal{S}^{t-1}}$
- 4: **else**
- 5: Use standard prior: $p_{\text{mixture}}(z) = \mathcal{N}(0, I)$
- 6: **end if**
- 7: Initialize: $\theta_i^t \leftarrow \theta^t, \phi_i^t \leftarrow \phi^t$
- 8: Initialize: $\mathcal{Z}_{\text{batch}} \leftarrow \emptyset$ (for collecting z values)
- 9: **for** local step $e = 1, \dots, E$ **do**
- 10: **for** batch $(s_A, s_B, y) \in \mathcal{D}_i$ **do**
- 11: Extract features: $h_{\text{chosen}}, h_{\text{rejected}} \leftarrow \text{LLM}(s_A, s_B)$
- 12: Compute difference: $\Delta h = h_{\text{chosen}} - h_{\text{rejected}}$ (or use choice logits)
- 13: Process: $f_i \leftarrow \text{FeatureExtractor}(\Delta h)$
- 14: Encode: $(\mu_i, \sigma_i^2) \leftarrow q_{\phi_i^t}(f_i)$
- 15: Sample: $z_i \sim \mathcal{N}(\mu_i, \sigma_i^2 I)$ via reparameterization trick
- 16: Collect: $\mathcal{Z}_{\text{batch}} \leftarrow \mathcal{Z}_{\text{batch}} \cup \{z_i\}$
- 17: Project: $\Delta \text{logits} \leftarrow \text{LatentProjection}(z_i)$
- 18: Condition: logits $\leftarrow$ logits_{base} + Δlogits
- 19: Compute reconstruction loss: $\mathcal{L}_{\text{recon}} \leftarrow -\log p_{\theta_i^t}(y | s_A, s_B, z_i)$
- 20: Compute KL divergence: $\mathcal{L}_{\text{KL}} \leftarrow \beta \cdot \mathbb{D}_{\text{KL}}(q_{\phi_i^t}(z | \cdot) || p_{\text{mixture}}(z))$
- 21: Compute orthogonal loss: $\mathcal{L}_{\text{ortho}} \leftarrow \lambda \cdot \mathcal{L}_{\text{orthogonal}}(z_i)$
- 22: Total loss: $\mathcal{L}_i \leftarrow \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}} + \mathcal{L}_{\text{ortho}}$
- 23: Update: θ_i^t, ϕ_i^t via SGD on $\mathcal{L}_i$ (only VPL components, base model frozen)
- 24: **end for**
- 25: **end for**
- 26: Compute average z distribution: $\bar{\mu}_i \leftarrow \text{mean}(\{\mu_i\})$, $\bar{\sigma}_i^2 \leftarrow \text{var}(\{z_i \in \mathcal{Z}_{\text{batch}}\})$
- 27: Send $(\theta_i^t, \phi_i^t, \bar{\mu}_i, \bar{\sigma}_i^2, |\mathcal{D}_i|)$ to server

orthogonality among all prototypes. The loss combines a pull term and an orthonormality constraint:

$$\mathcal{L}_{\text{orthogonal}}(z) = \|z - \mathbf{p}_{y_i^*}\|_2^2 + \gamma \cdot \|\mathbf{P}\mathbf{P}^T - \mathbf{I}_M\|_F^2, \quad (9)$$

where $\mathbf{P} \in \mathbb{R}^{M \times d}$ is the matrix of stacked prototypes. This mechanism forces latent representations into distinct orthogonal subspaces, effectively disentangling conflicting preferences (e.g., helpful vs. harmless).

4.3. Federated Variational Objective

During Stage 1, we aim to maximize the Evidence Lower Bound (ELBO) regularized by the orthogonal loss. The

local loss function for client i is:

$$\begin{aligned} \mathcal{L}_i(\theta, \phi) = & -\mathbb{E}_{z \sim q_\phi} \left[\underbrace{\sum_{(s_A, s_B, y) \in \mathcal{D}_i} \log p_\theta(y | s_A, s_B, z)}_{\mathcal{L}_{\text{recon}}} \right] \\ & + \underbrace{\beta \cdot \mathbb{D}_{KL}(q_\phi(z | \mathcal{D}_i) \| p_{\text{mixture}}^{(i)}(z))}_{\mathcal{L}_{\text{reg}} \text{ (Prior Matching)}} \\ & + \underbrace{\lambda \cdot \mathcal{L}_{\text{orthogonal}}(z)}_{\mathcal{L}_{\text{ortho}}}, \end{aligned} \quad (10)$$

where β controls KL regularization and λ weights the separation penalty. The first two terms constitute the negative ELBO, while the third enforces orthogonality.

4.4. Two-Stage Training Strategy

Finally, we describe the deployment pipeline, adopting a two-stage strategy (Wu et al., 2024) to handle heterogeneity efficiently.

Stage 1 (Federated Selector Training): We train the variational binary preference selector using the objective $\mathcal{L}_i(\theta, \phi)$ defined above. In this phase, each client learns a posterior $q_\phi(z | \mathcal{D}_i)$ and predicts choices conditioned on z_i . The preference prediction is performed via a Latent Conditional Reward Model:

$$\text{logits}(s_A, s_B | z_i) = \text{logits}_{\text{base}}(s_A, s_B) + f_\theta(z_i), \quad (11)$$

where f_θ is a learned linear projection mapping the latent vector z_i to logit adjustments. Clients leverage the mixture prior for knowledge transfer, enabling stable inference despite local data sparsity without exchanging raw data.

Stage 2 (Conditional RLHF): We perform Centralized RLHF (Rafailov et al., 2023) on the server. We employ DPO to train a policy conditioned on the inferred client context z (e.g., $z \sim q_i$). The converged selector from Stage 1 serves as the reward model, scoring generations as $\text{logits}(s_A, s_B | z)$. This decoupling avoids the prohibitive communication costs of federated generation and mitigates training instability caused by conflicting local gradients (Wu et al., 2024).

5. Experiments

5.1. Experimental Settings

Datasets and Non-IID Partition We evaluate our framework on the HH-RLHF dataset (Bai et al., 2022), which contains pairwise comparisons focused on helpfulness and harmlessness. To simulate a heterogeneous federated setting, we implement a strict Non-IID partition where clients are divided into two disjoint groups: 50% of clients exclusively hold preference pairs labeled for helpfulness, while the remaining 50% possess only harmlessness data. This partition

models a scenario where local preference data are highly heterogeneous and conflicting. We vary the total number of clients $K \in \{10, 50, 100\}$ and use fixed sampling rates to assess scalability.

Models We utilize two base models to validate performance across different scales: **Qwen-2 0.5B** (Yang et al., 2024) and **Gemma-2B** (Gemma Team et al., 2024). To ensure communication efficiency in the federated setting, both models are fine-tuned using LoRA (Hu et al., 2022).

Detailed training configurations and hyperparameter settings are provided in the Appendix.

Baselines We compare FedVPA-GP against three representative baselines:

- **FedDPO** (Ye et al., 2024): Standard federated DPO.
- **FedBiscuit** (Wu et al., 2024): A Federated preference alignment algorithm that trains light-weight binary selector through FL.
- **FedVPL**: A naive adaptation of VPL (Poddar et al., 2024) to FL using a fixed standard Gaussian prior $\mathcal{N}(0, I)$ without the orthogonal loss.

Evaluation Metrics Following standard benchmarks (Bai et al., 2022), we employ **GPT-4** (OpenAI, 2024) as an judge to evaluate the quality of responses generated by the fine-tuned models against a frozen baseline. We report the Win-rate (%) for both *Helpfulness* and *Harmlessness* on a held-out test set, assessing the model’s ability to satisfy conflicting user preferences.

5.2. Personalized Preference Alignment

Table 1 presents the GPT-4 win-rates of FedVPA-GP compared to state-of-the-art federated baselines on the HH-RLHF dataset across varying client scales.

Overcoming the Limits of Monolithic Reward Models

As hypothesized, baselines relying on monolithic reward models (FedDPO and FedBiscuit) struggle to reconcile conflicting preference objectives. As shown in Table 1, these methods often suffer from a severe trade-off: they tend to align the model towards harmlessness at the expense of helpfulness. This is particularly evident in the Qwen-2 experiments, where the helpfulness win-rate of baselines stagnates or even decreases as the focus shifts to harmlessness. In contrast, FedVPA-GP effectively disentangles these conflicting heterogeneous preferences by conditioning the reward model on client-specific latent variables. Consequently, our method achieves a Pareto improvement, securing significantly higher win-rates in both helpfulness and

Table 1. **Main Results on HH-RLHF.** We report the GPT-4 Win-rate (%) across varying client counts ($N \in \{10, 50, 100\}$). Shaded rows indicate our proposed method, **FedVPA-GP**, which consistently achieves the best trade-off between helpfulness and harmlessness.

MODEL	METHOD	10 CLIENTS		50 CLIENTS		100 CLIENTS	
		HELPFUL	HARMLESS	HELPFUL	HARMLESS	HELPFUL	HARMLESS
QWEN 2	FEDDPO	48.12 $\pm$ 1.52	77.34 $\pm$ 2.41	43.05 $\pm$ 2.15	69.22 $\pm$ 2.85	41.48 $\pm$ 2.32	67.15 $\pm$ 2.64
	FEDBISCUIT	48.85 $\pm$ 1.41	75.12 $\pm$ 2.28	44.21 $\pm$ 1.98	71.45 $\pm$ 2.61	42.33 $\pm$ 2.11	69.42 $\pm$ 2.45
	FEDVPL	62.24 $\pm$ 1.25	84.56 $\pm$ 1.95	54.18 $\pm$ 1.72	78.12 $\pm$ 2.12	53.05 $\pm$ 1.88	77.34 $\pm$ 2.21
	FEDVPA-GP	66.45 $\pm$ 1.12	89.21 $\pm$ 1.68	58.32 $\pm$ 1.45	84.05 $\pm$ 1.94	55.18 $\pm$ 1.55	82.31 $\pm$ 2.05
GEMMA-2B	FEDDPO	52.34 $\pm$ 1.75	83.12 $\pm$ 2.55	44.15 $\pm$ 2.31	78.45 $\pm$ 2.92	41.22 $\pm$ 2.58	75.33 $\pm$ 3.12
	FEDBISCUIT	51.65 $\pm$ 1.58	82.45 $\pm$ 2.32	46.21 $\pm$ 2.05	78.12 $\pm$ 2.74	43.44 $\pm$ 2.22	76.05 $\pm$ 2.88
	FEDVPL	66.82 $\pm$ 1.34	89.15 $\pm$ 2.05	56.41 $\pm$ 1.84	84.34 $\pm$ 2.31	53.25 $\pm$ 1.95	80.42 $\pm$ 2.45
	FEDVPA-GP	73.21 $\pm$ 1.15	96.34 $\pm$ 1.75	64.48 $\pm$ 1.52	95.12 $\pm$ 2.05	60.15 $\pm$ 1.68	92.45 $\pm$ 2.15

Table 2. **Unseen Client Generalization Results.** We report the GPT-4 Win-rate (%) for seen and unseen clients. Shaded rows indicate our proposed method.

Method	Seen		Unseen	
	Helpful	Harmless	Helpful	Harmless
FedDPO	46.35	78.62	47.27	79.15
FedBiscuit	47.32	79.25	47.62	78.42
FedVPL	56.23	83.82	49.25	75.21
FedVPA-GP	65.28	94.25	63.16	91.23

harmlessness compared to all baselines, demonstrating the efficacy of personalization in satisfying diverse user needs.

Robustness to Heterogeneity and Data Sparsity The experimental results also highlight the challenge of scaling in federated settings. As the number of clients increases, the number of data each local client possess becomes increasingly sparse and the aggregate distribution more heterogeneous. Table 1 demonstrates that the performance of baselines, and even the naive FedVPL, deteriorates notably under these conditions. While monolithic approaches struggle to maintain performance amidst this increased noise, FedVPA-GP exhibits robustness. By leveraging the Federated Mixture Prior to share distributional knowledge without sharing raw data, our approach maintains consistent and high alignment performance even in large-scale settings with high data sparsity, validating its stability in decentralized environments.

5.3. Analysis

Analysis of Latent Space Disentanglement To understand how the model represents conflicting preferences, Figure 3 visualizes the evolution of the latent preference distribution (z) of 5 clients preferring helpfulness and 5 clients

preferring harmlessness using t-SNE (van der Maaten & Hinton, 2008). Red and blue points correspond to latent z inferred from clients prioritizing harmlessness and helpfulness, respectively. As observed in the top row, the baseline FedVPL suffers from posterior collapse, where the distributions for these distinct preference types remain entangled and non-separable throughout the training process. In contrast, FedVPA-GP demonstrates a clear trajectory towards disentanglement. Driven by the Federated Mixture Prior and Orthogonal Loss, our method progressively structures the latent space, resulting in a sharp separation between the two preference types. This structured latent topology confirms that the model successfully learns to distinguish between conflicting user intents, enabling dynamic adaptation to local contexts.

Generalization to Unseen Clients To evaluate the robustness of our framework against new users, we conducted an experiment with 20 clients, equally divided into helpfulness and harmlessness clusters. We utilized a hold-out strategy where 5 clients from each cluster were used for training (Seen), while the remaining 5 clients from each cluster were reserved for evaluation (Unseen). For the variational approaches (FedVPL and FedVPA-GP), we performed variational inference on the unseen clients’ local datasets to estimate their latent preference vectors z without updating the model parameters. As shown in Table 2, monolithic baselines like FedDPO and FedBiscuit exhibit consistent performance across seen and unseen groups, but their overall win-rates are limited due to their inability to model personalization. In contrast, FedVPL suffers a significant performance degradation on unseen clients, indicating a failure to generalize the latent preference structure. However, FedVPA-GP demonstrates stability, maintaining high win-rates on unseen clients that are comparable to the seen clients. This result suggests that our proposed mixture prior and orthogonal regularization enable the model to learn a

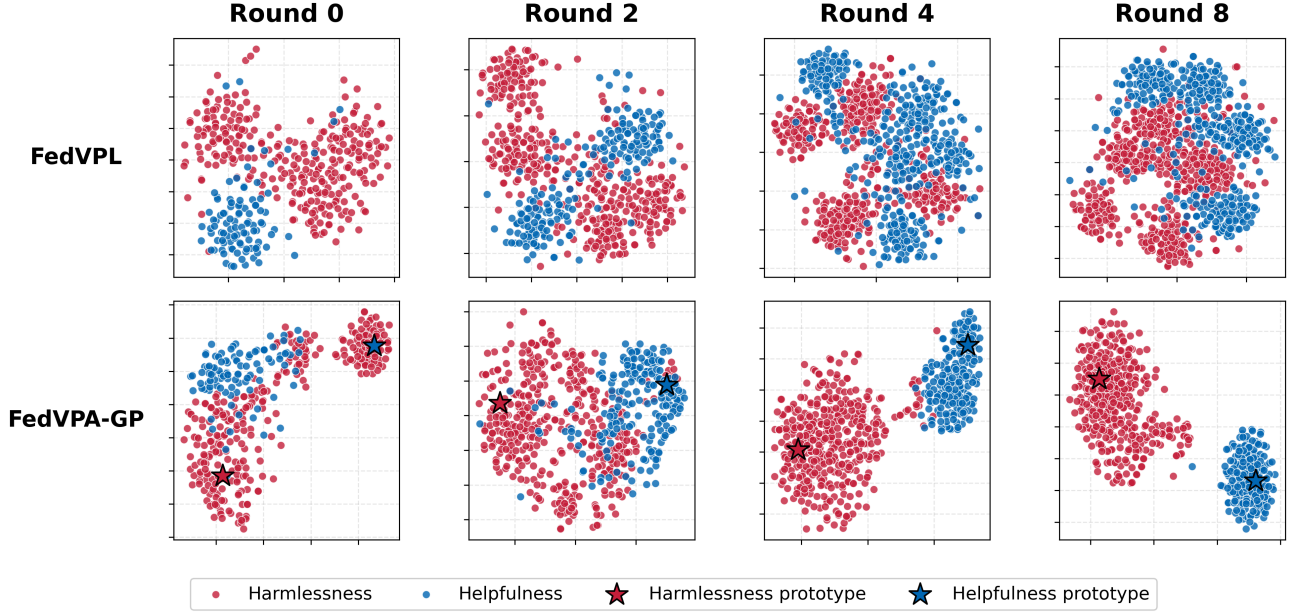


Figure 3. Evolution of client-specific latent preference distributions (z) across training rounds for **FedVPL** (top row) and **FedVPA-GP** (bottom row). Points are colored by preference type: red (harmless) and blue (helpfulness). Star markers (*) indicate orthogonal prototypes in FedVPA-GP. FedVPA-GP achieves better separation between preference types compared to FedVPL.

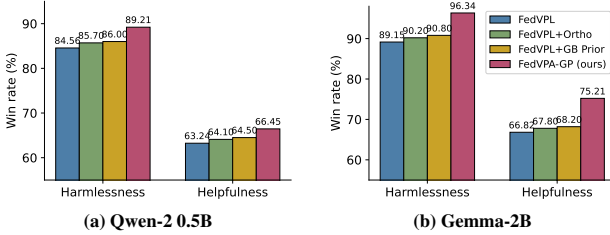


Figure 4. Ablation study: helpfulness and harmless win rate (%). (a) Qwen-2 0.5B. (b) Gemma-2B. Methods: FedVPL, FedVPL+Ortho, FedVPL+GB Prior, FedVPA-GP.

semantically meaningful and continuous latent space, allowing it to effectively capture and condition on the preferences of novel users via simple inference.

5.4. Ablation Study

We analyze the impact of our key components in Figure 4. Adding the Orthogonal Loss (FedVPL+Ortho) consistently improves both metrics by preventing latent overlap, thereby mitigating posterior collapse. Meanwhile, the Federated Mixture Prior (FedVPL+GB Prior) stabilizes training against data sparsity by leveraging the aggregate population distribution as a dynamic prior. Ultimately, the full FedVPA-GP framework achieves superior performance, demonstrating that the Mixture Prior ensures robust learning while the Orthogonal Loss enforces semantic disentanglement, yielding the best trade-off between conflicting preferences.

6. Conclusion

In this paper, we present **FedVPA-GP**, a framework designed to learn distributed preferences with the inherently pluralistic nature of human values. We identified that existing federated alignment methods suffer from a "monolithic" limitation, enforcing a non-existent consensus that averages out conflicting preferences. Moreover, we highlighted that applying variational personalization to decentralized settings is non-trivial, as the data scarcity and heterogeneity of isolated clients typically induce posterior collapse.

To address these fundamental challenges, we introduced a Federated Mixture Prior and an Orthogonal Loss. By leveraging the aggregate population distribution as a dynamic prior, our approach effectively bridges the gap between local isolation and global context, stabilizing variational inference without sharing raw data. Empirical results on the HH-RLHF dataset demonstrate that FedVPA-GP significantly outperforms monolithic baselines, successfully disentangling conflicting user intents within a structured latent space. Crucially, our framework generalizes to unseen clients, allowing adaptation to novel users via inference alone.

Our work provides a robust foundation for personalized, privacy-preserving LLM alignment. Future directions include extending this framework to capture more granular, multi-dimensional preference attributes and investigating its scalability in large-scale cross-device settings with extreme communication constraints.

7. Impact Statement

This work advances privacy-preserving AI by enabling the alignment of LLMs with diverse user preferences without centralizing sensitive data. By moving away from monolithic value systems, our framework respects the inherent pluralism of human values, allowing models to adapt to conflicting objectives like helpfulness and harmlessness. However, extreme personalization carries the risk of creating “filter bubbles” where models might reinforce harmful user biases. While our method explicitly models harmlessness to mitigate this, future deployment must carefully balance personalization with robust safety guardrails to ensure ethical boundaries are maintained.

References

Alemi, A., Poole, B., Fischer, I., Dillon, J., Saourous, R. A., and Murphy, K. Fixing a broken ELBO. In *International Conference on Machine Learning (ICML)*, pp. 159–168. PMLR, 2018.

Azar, M. G., Rowland, M., Piot, B., Guo, D., Calandriello, D., Valko, M., and Munos, R. A general theoretical paradigm to understand learning from human preferences. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2024.

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., Das-Sarma, N., Drain, D., et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Bowman, S. R., Vilnis, L., Vinyals, O., Dai, A., Jozefowicz, R., and Bengio, S. Generating sentences from a continuous space. In *Proceedings of the 20th SIGNLL Conference on Computational Natural Language Learning (CoNLL)*, pp. 10–21, 2016.

Bradley, R. A. and Terry, M. E. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., et al. Extracting training data from large language models. In *USENIX Security Symposium*, volume 6, 2021.

Christiano, P. F., Leike, J., Brown, T., Milani, M., Amodei, D., and Amodei, D. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Dong, H., Xiong, W., Goyal, D., Pan, R., Diao, S., Zhang, J., Shum, K., and Zhang, T. Steerlm: Attribute conditioned sft as an (user-steerable) alternative to rlhf. *arXiv preprint arXiv:2310.05344*, 2023.

Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., and Kiela, D. Kto: Model alignment as prospect theoretic optimization. In *International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=Duqy5E9nF8>.

European Parliament and Council of the European Union. Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data. *Official Journal of the European Union*, L119:1–88, 2016.

Gemma Team, Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M. S., Love, J., et al. Gemma: Open models. *arXiv preprint arXiv:2403.08295*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations (ICLR)*, 2017.

Jang, J., Kim, S., Ye, S., Kim, D., Yao, L., Agrawal, A., and Seo, M. Personalized soups: Personalized large language model alignment via post-hoc parameter merging. *arXiv preprint arXiv:2310.11600*, 2023.

Kairouz, P., McMahan, H. B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A. N., et al. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning*, 14(1–2):1–210, 2021.

Kingma, D. P. and Welling, M. Auto-encoding variational bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

Li, H., Nguyen, M., and Pimentel-Alarcón, D. Preventing collapse in contrastive learning with orthonormal prototypes (clop), 2024. URL <https://arxiv.org/abs/2403.18699>.

Luce, R. D. *Individual choice behavior: A theoretical analysis*. John Wiley & Sons, 1959.

McMahan, B., Moore, E., Ramage, D., Hampson, S., and Arcas, B. A. y. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics (AISTATS)*, pp. 1273–1282, 2017.

OpenAI. Gpt-4o system card, 2024. URL <https://openai.com/index/gpt-4o-system-card/>. OpenAI.

- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pp. 27730–27744, 2022.
- Poddar, S., Wan, Y., Ivison, H., Gupta, A., and Jaques, N. Personalizing reinforcement learning from human feedback with variational preference learning. *arXiv preprint arXiv:2408.10075*, 2024.
- Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Rame, A., Ahuja, K., Zhang, J., Rivolle, Yannick and Lopez-Paz, D., and Bottou, L. Rewarded soups: Towards pareto-optimal alignment by interpolating weights. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.
- Santurkar, S., Durmus, E., Ladhak, F., Lee, C., Liang, P., and Hashimoto, T. Whose opinions do language models reflect? In *International Conference on Machine Learning (ICML)*, 2023.
- Saxe, A. M., McClelland, J. L., and Ganguli, S. Exact solutions to the nonlinear dynamics of learning in deep linear neural networks. *arXiv preprint arXiv:1312.6120*, 2013.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shirali, A., Nasr-Esfahany, A., Alomar, A., Mirtaheeri, P., Abebe, R., and Procaccia, A. D. Direct alignment with heterogeneous preferences. *arXiv preprint arXiv:2502.16320*, 2025.
- van der Maaten, L. and Hinton, G. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9(11): 2579–2605, 2008.
- Wu, F., Liu, X., Wang, H., Wang, X., and Gao, J. Towards federated rlhf with aggregated client preference for llms. *arXiv preprint arXiv:2407.03038*, 2024.
- Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C., Li, C., Li, C., Liu, D., Huang, F., et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10670*, 2024.
- Ye, R., Wang, W., Chai, J., Li, D., Li, Z., Xu, Y., Du, Y., Wang, Y., and Chen, S. Openfedllm: Training large language models on decentralized private data via federated learning. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 6137–6147, 2024.
- Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., and Christiano, P. F. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

A. Mathematical Proofs

This section provides detailed mathematical proofs and derivations for key components of our method.

A.1. KL Divergence with Mixture Prior

A.1.1. DEFINITION AND COMPUTATION

Theorem 1 (KL Divergence with Mixture Prior): The KL divergence between a posterior distribution $q_i(z) = \mathcal{N}(z; \mu_i, \sigma_i^2 I)$ and a mixture prior $p_{\text{mixture}}(z) = \sum_{j=1}^{|S|} w_j \cdot \mathcal{N}(z; \mu_j, \sigma_j^2 I)$ is given by:

$$\mathbb{D}_{KL}(q_i \parallel p_{\text{mixture}}) = \mathbb{E}_{z \sim q_i} [\log q_i(z) - \log p_{\text{mixture}}(z)], \quad (12)$$

where S is the set of participating clients from the previous round and w_j are normalized weights (typically sample-size weighted: $w_j = n_j / \sum_{k \in S} n_k$).

Proof: By definition of KL divergence:

$$\begin{aligned} \mathbb{D}_{KL}(q_i \parallel p_{\text{mixture}}) &= \int q_i(z) \log \frac{q_i(z)}{p_{\text{mixture}}(z)} dz \\ &= \int q_i(z) [\log q_i(z) - \log p_{\text{mixture}}(z)] dz \\ &= \mathbb{E}_{z \sim q_i} [\log q_i(z) - \log p_{\text{mixture}}(z)]. \end{aligned} \quad (13)$$

For a multivariate Gaussian posterior $q_i(z) = \mathcal{N}(z; \mu_i, \sigma_i^2 I)$ with dimension d , we have:

$$\log q_i(z) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^d \log \sigma_{i,j}^2 - \frac{1}{2} \sum_{j=1}^d \frac{(z_j - \mu_{i,j})^2}{\sigma_{i,j}^2}. \quad (14)$$

For the mixture prior, we compute:

$$\log p_{\text{mixture}}(z) = \log \left[\sum_{j=1}^{|S|} w_j \cdot \mathcal{N}(z; \mu_j, \sigma_j^2 I) \right]. \quad (15)$$

Using Monte Carlo estimation with batch size B :

$$\mathbb{D}_{KL} \approx \frac{1}{B} \sum_{b=1}^B [\log q_i(z^{(b)}) - \log p_{\text{mixture}}(z^{(b)})], \quad (16)$$

where $z^{(b)} \sim q_i$ is sampled using the reparameterization trick:

$$z^{(b)} = \mu_i + \sigma_i \odot \epsilon^{(b)}, \quad \epsilon^{(b)} \sim \mathcal{N}(0, I), \quad (17)$$

and $\sigma_i = \exp(0.5 \cdot \log \sigma_i^2)$ with $\odot$ denoting element-wise multiplication.

□

A.1.2. LOG-SUM-EXP TRICK FOR NUMERICAL STABILITY

Theorem 2 (Log-Sum-Exp Trick): For numerical stability when computing $\log p_{\text{mixture}}(z)$, we use the log-sum-exp trick:

$$\log \left(\sum_{i=1}^N \exp(a_i) \right) = \max_i a_i + \log \left(\sum_{i=1}^N \exp(a_i - \max_i a_i) \right). \quad (18)$$

Proof: We factor out the maximum term:

$$\sum_{i=1}^N \exp(a_i) = \exp(\max_i a_i) \cdot \sum_{i=1}^N \exp(a_i - \max_i a_i). \quad (19)$$

Taking the logarithm of both sides:

$$\log \left(\sum_{i=1}^N \exp(a_i) \right) = \max_i a_i + \log \left(\sum_{i=1}^N \exp(a_i - \max_i a_i) \right). \quad (20)$$

Since $\exp(a_i - \max_i a_i) \in [0, 1]$ for all i , this formulation is numerically stable and prevents overflow/underflow.

For the mixture prior, we apply this trick with:

$$a_j = \log w_j + \log \mathcal{N}(z; \mu_j, \sigma_j^2 I), \quad (21)$$

where:

$$\log \mathcal{N}(z; \mu_j, \sigma_j^2 I) = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \sum_{k=1}^d \log \sigma_{j,k}^2 - \frac{1}{2} \sum_{k=1}^d \frac{(z_k - \mu_{j,k})^2}{\sigma_{j,k}^2}. \quad (22)$$

□

A.2. Reparameterization Trick and Gradient Flow

Theorem 3 (Reparameterization Trick Gradient Flow): Using the reparameterization trick, gradients with respect to z flow to μ and $\log \sigma^2$.

Proof: The reparameterization trick expresses the random variable z as a deterministic function of parameters and noise:

$$z = \mu + \epsilon \odot \exp(0.5 \cdot \log \sigma^2), \quad \epsilon \sim \mathcal{N}(0, I), \quad (23)$$

where $\sigma = \exp(0.5 \cdot \log \sigma^2)$.

The partial derivatives are:

$$\frac{\partial z}{\partial \mu} = I \quad (\text{identity matrix}), \quad (24)$$

$$\frac{\partial z}{\partial \log \sigma^2} = \epsilon \odot \exp(0.5 \cdot \log \sigma^2) \odot 0.5. \quad (25)$$

By the chain rule, for any function $f(z)$:

$$\frac{\partial f(z)}{\partial \mu} = \frac{\partial f(z)}{\partial z} \cdot \frac{\partial z}{\partial \mu} = \frac{\partial f(z)}{\partial z}, \quad (26)$$

$$\frac{\partial f(z)}{\partial \log \sigma^2} = \frac{\partial f(z)}{\partial z} \cdot \frac{\partial z}{\partial \log \sigma^2} = \frac{\partial f(z)}{\partial z} \cdot \epsilon \odot \sigma \odot 0.5. \quad (27)$$

This ensures that gradients can flow through the sampling operation, enabling end-to-end training of the variational encoder.

□

A.3. Orthogonal Loss Formulation

A.3.1. PULL LOSS

The pull loss encourages latent representations z to align with their assigned prototypes:

$$\mathcal{L}_{\text{pull}}(z) = \|z - \mathbf{p}_{y_i^*}\|_2^2, \quad (28)$$

where $\mathbf{p}_{y_i^*}$ is the prototype assigned to client i based on the server's orthogonal label $y_i^* \in \{0, 1\}$.

A.3.2. ORTHONORMALITY CONSTRAINT

To maintain orthogonality between prototypes, we enforce an orthonormality constraint:

$$\mathcal{L}_{\text{orthonorm}} = \|\mathbf{P}^T \mathbf{P} - \mathbf{I}\|_F^2, \quad (29)$$

where $\mathbf{P} = [\mathbf{p}_0, \mathbf{p}_1]^T$ is the prototype matrix and $\|\cdot\|_F$ denotes the Frobenius norm.

This constraint ensures that $\mathbf{p}_0^T \mathbf{p}_1 = 0$ (orthogonality) and $\|\mathbf{p}_0\|_2 = \|\mathbf{p}_1\|_2 = 1$ (normalization).

A.3.3. TOTAL ORTHOGONAL LOSS

The combined orthogonal loss is:

$$\mathcal{L}_{\text{orthogonal}}(z) = \lambda \cdot \mathcal{L}_{\text{pull}}(z) + \gamma \cdot \mathcal{L}_{\text{orthonorm}}, \quad (30)$$

where $\lambda = 1.0$ (orthogonal weight) and $\gamma = 0.1$ (orthonorm weight) are hyperparameters.

This loss encourages latent representations to cluster around their assigned prototypes while ensuring that different preference types occupy orthogonal subspaces, thereby suppressing general features that are not preference-specific and preventing neural collapse.

A.4. Evidence Lower Bound (ELBO) Derivation

Theorem 4 (ELBO Derivation): The Evidence Lower Bound (ELBO) for variational inference is:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} [\log p_\theta(\mathcal{D}_i | z)] - \beta \mathbb{D}_{KL}(q_\phi(z | \mathcal{D}_i) \| p(z)), \quad (31)$$

where β is a regularization coefficient.

Proof: We start with the log-likelihood of the data:

$$\log p_\theta(\mathcal{D}_i) = \log \int p_\theta(\mathcal{D}_i | z) p(z) dz. \quad (32)$$

Introducing the variational posterior $q_\phi(z | \mathcal{D}_i)$:

$$\begin{aligned} \log p_\theta(\mathcal{D}_i) &= \log \int q_\phi(z | \mathcal{D}_i) \cdot \frac{p_\theta(\mathcal{D}_i | z) p(z)}{q_\phi(z | \mathcal{D}_i)} dz \\ &= \log \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} \left[\frac{p_\theta(\mathcal{D}_i | z) p(z)}{q_\phi(z | \mathcal{D}_i)} \right]. \end{aligned} \quad (33)$$

Applying Jensen's inequality (since log is concave):

$$\begin{aligned}
 \log p_\theta(\mathcal{D}_i) &\geq \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} \left[\log \frac{p_\theta(\mathcal{D}_i | z)p(z)}{q_\phi(z | \mathcal{D}_i)} \right] \\
 &= \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} [\log p_\theta(\mathcal{D}_i | z)] + \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} \left[\log \frac{p(z)}{q_\phi(z | \mathcal{D}_i)} \right] \\
 &= \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} [\log p_\theta(\mathcal{D}_i | z)] - \mathbb{D}_{KL}(q_\phi(z | \mathcal{D}_i) \| p(z)).
 \end{aligned} \tag{34}$$

Introducing the regularization coefficient β to control the strength of the KL regularization:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(z|\mathcal{D}_i)} [\log p_\theta(\mathcal{D}_i | z)] - \beta \mathbb{D}_{KL}(q_\phi(z | \mathcal{D}_i) \| p(z)). \tag{35}$$

The first term is the reconstruction loss (preference alignment), and the second term is the regularization (prior matching). Maximizing the ELBO is equivalent to minimizing the negative ELBO:

$$\mathcal{L}_i(\theta, \phi) = -\text{ELBO}(\mathcal{D}_i) = -\mathbb{E}_{q_\phi} [\log p_\theta(\mathcal{D}_i | z)] + \beta \mathbb{D}_{KL}(q_\phi(z | \mathcal{D}_i) \| p(z)). \tag{36}$$

□

A.5. Standard Gaussian Prior KL Divergence

For the baseline VPL ablation, we use a standard Gaussian prior $p(z) = \mathcal{N}(0, I)$. The KL divergence has a closed-form expression:

Theorem 5 (Standard Gaussian Prior KL): For $q_i(z) = \mathcal{N}(z; \mu_i, \sigma_i^2 I)$ and $p(z) = \mathcal{N}(0, I)$, the KL divergence is:

$$\mathbb{D}_{KL}(q_i \| \mathcal{N}_0) = \frac{1}{2} \sum_{j=1}^d (\mu_{i,j}^2 + \sigma_{i,j}^2 - \log \sigma_{i,j}^2 - 1). \tag{37}$$

Proof: For two multivariate Gaussians, the KL divergence is:

$$\mathbb{D}_{KL}(\mathcal{N}(\mu_1, \Sigma_1) \| \mathcal{N}(\mu_2, \Sigma_2)) = \frac{1}{2} \left[\text{tr}(\Sigma_2^{-1} \Sigma_1) + (\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1) - d + \log \frac{|\Sigma_2|}{|\Sigma_1|} \right]. \tag{38}$$

For $q_i = \mathcal{N}(\mu_i, \sigma_i^2 I)$ and $p = \mathcal{N}(0, I)$:

$$\begin{aligned}
 \mathbb{D}_{KL}(q_i \| \mathcal{N}_0) &= \frac{1}{2} \left[\text{tr}(I^{-1} \cdot \sigma_i^2 I) + (0 - \mu_i)^T I^{-1} (0 - \mu_i) - d + \log \frac{|I|}{|\sigma_i^2 I|} \right] \\
 &= \frac{1}{2} [\text{tr}(\sigma_i^2 I) + \mu_i^T \mu_i - d - \log |\sigma_i^2 I|] \\
 &= \frac{1}{2} \left[\sum_{j=1}^d \sigma_{i,j}^2 + \sum_{j=1}^d \mu_{i,j}^2 - d - \sum_{j=1}^d \log \sigma_{i,j}^2 \right] \\
 &= \frac{1}{2} \sum_{j=1}^d (\mu_{i,j}^2 + \sigma_{i,j}^2 - \log \sigma_{i,j}^2 - 1).
 \end{aligned} \tag{39}$$

□

A.6. Gumbel-Softmax for Differentiable Prior Sampling

For sampling from the mixture prior (used in visualization and generation), we employ Gumbel-Softmax relaxation (Jang et al., 2017) with temperature $\tau = 1.0$ to enable differentiable sampling.

Component probabilities:

$$\alpha_j = \frac{\exp((\log w_j + g_j)/\tau)}{\sum_{k=1}^{|S|} \exp((\log w_k + g_k)/\tau)}, \quad (40)$$

where $g_j \sim \text{Gumbel}(0, 1)$ are independent Gumbel random variables.

Sampling:

$$z_{\text{prior}} = \sum_{j=1}^{|S|} \alpha_j \cdot z_j, \quad \text{where } z_j \sim \mathcal{N}(\mu_j, \sigma_j^2 I). \quad (41)$$

As $\tau \rightarrow 0$, this approaches categorical sampling (hard assignment), while $\tau > 0$ provides a smooth, differentiable approximation.

A.7. Gradient Computation for Mixture Prior

Theorem 6 (Gradient of KL with Respect to Posterior Parameters): The gradient of the KL divergence with respect to posterior parameters μ_i and $\log \sigma_i^2$ is:

$$\frac{\partial \mathbb{D}_{KL}}{\partial \mu_i} = \frac{1}{B} \cdot \left(\frac{z - \mu_i}{\sigma_i^2} + \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2} \right), \quad (42)$$

$$\frac{\partial \mathbb{D}_{KL}}{\partial \log \sigma_i^2} = \frac{1}{B} \cdot \left(-\frac{1}{2} + \frac{(z - \mu_i)^2}{2\sigma_i^2} + \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2} \cdot \epsilon \odot \sigma_i \odot 0.5 \right), \quad (43)$$

where $\text{softmax}_j = \exp(a_j - \max_k a_k) / \sum_{k=1}^{|S|} \exp(a_k - \max_k a_k)$ with $a_j = \log w_j + \log \mathcal{N}(z; \mu_j, \sigma_j^2 I)$.

Proof: The KL divergence is $\mathbb{D}_{KL} = \frac{1}{B} \sum_{b=1}^B [\log q_i(z^{(b)}) - \log p_{\text{mixture}}(z^{(b)})]$, where $z^{(b)} = \mu_i + \sigma_i \odot \epsilon^{(b)}$.

The gradient has two paths:

Path 1: Through $\log q_i(z)$:

$$\frac{\partial \mathbb{D}_{KL}}{\partial \log q_i(z)} = \frac{1}{B}, \quad (44)$$

$$\frac{\partial \log q_i(z)}{\partial \mu_i} = \frac{z - \mu_i}{\sigma_i^2}, \quad (45)$$

$$\frac{\partial \log q_i(z)}{\partial \log \sigma_i^2} = -\frac{1}{2} + \frac{(z - \mu_i)^2}{2\sigma_i^2}. \quad (46)$$

By chain rule:

$$\left. \frac{\partial \mathbb{D}_{KL}}{\partial \mu_i} \right|_{\text{Path 1}} = \frac{1}{B} \cdot \frac{z - \mu_i}{\sigma_i^2}, \quad (47)$$

$$\left. \frac{\partial \mathbb{D}_{KL}}{\partial \log \sigma_i^2} \right|_{\text{Path 1}} = \frac{1}{B} \cdot \left(-\frac{1}{2} + \frac{(z - \mu_i)^2}{2\sigma_i^2} \right). \quad (48)$$

Path 2: Through $\log p_{\text{mixture}}(z)$:

$$\frac{\partial \mathbb{D}_{KL}}{\partial \log p_{\text{mixture}}(z)} = -\frac{1}{B}, \quad (49)$$

$$\frac{\partial \log p_{\text{mixture}}(z)}{\partial z} = \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{-(z - \mu_j)}{\sigma_j^2} = -\sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2}. \quad (50)$$

By the reparameterization trick:

$$\frac{\partial z}{\partial \mu_i} = I, \quad (51)$$

$$\frac{\partial z}{\partial \log \sigma_i^2} = \epsilon \odot \sigma_i \odot 0.5. \quad (52)$$

By chain rule:

$$\frac{\partial \mathbb{D}_{KL}}{\partial z} = -\frac{1}{B} \cdot \left(-\sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2} \right) = \frac{1}{B} \cdot \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2}, \quad (53)$$

$$\left. \frac{\partial \mathbb{D}_{KL}}{\partial \mu_i} \right|_{\text{Path 2}} = \frac{\partial \mathbb{D}_{KL}}{\partial z} \cdot \frac{\partial z}{\partial \mu_i} = \frac{1}{B} \cdot \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2}, \quad (54)$$

$$\left. \frac{\partial \mathbb{D}_{KL}}{\partial \log \sigma_i^2} \right|_{\text{Path 2}} = \frac{\partial \mathbb{D}_{KL}}{\partial z} \cdot \frac{\partial z}{\partial \log \sigma_i^2} = \frac{1}{B} \cdot \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2} \cdot \epsilon \odot \sigma_i \odot 0.5. \quad (55)$$

Combining both paths:

$$\frac{\partial \mathbb{D}_{KL}}{\partial \mu_i} = \frac{1}{B} \cdot \frac{z - \mu_i}{\sigma_i^2} + \frac{1}{B} \cdot \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2}, \quad (56)$$

$$\frac{\partial \mathbb{D}_{KL}}{\partial \log \sigma_i^2} = \frac{1}{B} \cdot \left(-\frac{1}{2} + \frac{(z - \mu_i)^2}{2\sigma_i^2} \right) + \frac{1}{B} \cdot \sum_{j=1}^{|S|} \text{softmax}_j \cdot \frac{z - \mu_j}{\sigma_j^2} \cdot \epsilon \odot \sigma_i \odot 0.5. \quad (57)$$

□

A.8. Prior Parameter Non-Differentiability

Theorem 7 (Prior Parameter Non-Differentiability): The mixture prior parameters μ_j, σ_j^2, w_j from other clients do not receive gradients.

Proof: Prior parameters are fixed values from other clients' distributions, used as constants in the forward pass:

$$p_{\text{mixture}}(z) = \sum_{j=1}^{|S|} w_j \cdot \mathcal{N}(z; \mu_j, \sigma_j^2 I), \quad (58)$$

where μ_j, σ_j^2, w_j are fixed (not part of the computation graph for the current client).

In the gradient computation:

$$\frac{\partial \mathbb{D}_{KL}}{\partial \mu_j} = \frac{\partial}{\partial \mu_j} \mathbb{E}_{z \sim q_i} [\log q_i(z) - \log p_{\text{mixture}}(z)] = 0, \quad (59)$$

since μ_j is not a leaf node in the computation graph and autograd does not compute gradients for it.

□

B. Hyperparameter Details

B.1. Hyperparameter Settings

Table 3 provides the final hyperparameter values used in our experiments.

Table 3. Final hyperparameter settings for selector training (Stage 1) and RL training (Stage 2).

Parameter	Selector Training	RL Training
Learning rate	10^{-4} (Gemma-2B), 10^{-5} (Qwen-2 0.5B)	10^{-4} (Gemma-2B), 10^{-5} (Qwen-2 0.5B)
Batch size	8–16 (model-dependent)	1
Gradient accumulation steps	4	32 (Qwen-2), 4 (Gemma-2B)
Local update steps	30	30
Total rounds	50	50
KL weight (β)	0.1	–
Orthogonal loss weight (λ)	1.0	–
Orthonorm weight (γ)	0.1	–
Gumbel-Softmax temperature (τ)	1.0	–
Prototype scale (s)	5.0	–
Latent dimension (d)	32	–
LoRA rank (r)	8	8
LoRA alpha (α)	16	16
LoRA dropout (p)	0.05	0.05
Reward coefficient	–	0.1
Max prompts for generation	–	50
Generation batch size	–	3
Max samples for reward	–	30

C. Experimental Settings

C.1. Dataset Details

C.1.1. HH-RLHF DATASET

We use the **HH-RLHF** (Helpful and Harmless from Human Feedback) dataset (Bai et al., 2022), which contains pairwise preference comparisons along two axes: helpfulness and harmlessness.

Data splits:

- Train: 80%
- Validation: 10%
- Test: 10%

Client configurations:

- Number of clients: $K \in \{10, 50, 100\}$
- Sampling rates per round:
 - $K = 10$: 5 clients per round
 - $K \in \{50, 100\}$: 10 clients per round

Data characteristics:

- Data type: Pairwise preference comparisons
- Preference axes: Helpfulness, Harmlessness
- Each sample: (s_A, s_B, y) where $y \in \{0, 1\}$ indicates preference

C.2. Model Details

C.2.1. BASE LANGUAGE MODELS

We conduct experiments using two base language models:

- **Qwen-2 0.5B**: A compact 0.5 billion parameter model from the Qwen-2 family (Yang et al., 2024), suitable for resource-constrained federated environments.
- **Gemma-2B**: A 2 billion parameter model from Google’s Gemma family (Gemma Team et al., 2024), providing a larger model baseline for comparison.

C.2.2. FINE-TUNING CONFIGURATION

Both models are fine-tuned using LoRA (Low-Rank Adaptation) (Hu et al., 2022) to enable parameter-efficient fine-tuning in federated settings.

LoRA parameters:

- LoRA rank: $r = 8$
- LoRA alpha: $\alpha = 16$
- Dropout rate: $p = 0.05$

Training configuration:

- During federated selector training: Base LLM is frozen; only VPL components (feature extractor, variational encoder, latent projection) and LoRA adapters are updated.
- During RL training: All parameters (base LLM + adapters) are trainable.

C.3. Evaluation Settings

C.3.1. WINRATE EVALUATION

We evaluate the final policy using **GPT-4 Win-rate** (Bai et al., 2022):

- **Judge**: GPT-4 or gpt-4o-mini (for cost efficiency)
- **Comparison**: Fine-tuned model response vs baseline (frozen) model response
- **Evaluation samples**: Up to 30 samples per evaluation (for efficiency)
- **Metrics**:
 - **Helpful Win-rate (%)**: Percentage of responses where fine-tuned model is more helpful
 - **Harmless Win-rate (%)**: Percentage of responses where fine-tuned model is more harmless

C.3.2. REWARD MODEL EVALUATION

We also evaluate using reward model scores:

- **Harmlessness score**: Reward model’s harmlessness score (higher is better)
- **Helpfulness score**: Reward model’s helpfulness score (higher is better)
- **Evaluation samples**: Up to 100 samples

C.4. Baseline Methods

We compare our proposed **FedVPA-GP** with the following baselines:

- **FedDPO**: Federated Direct Preference Optimization (Ye et al., 2024), which aggregates gradients for a single global policy without personalized reward modeling.
- **FedBiscuit**: Federated learning with multiple LoRA adapters ($U = 3$) for coarse-grained personalization (Wu et al., 2024).
- **FedVPL**: Our naive adaptation of VPL (Poddar et al., 2024) to FL—same latent selector with FedAvg, but using fixed Gaussian prior $\mathcal{N}(0, I)$ and no orthogonal loss.
- **FedVPA-GP** (ours): Our full method with federated mixture prior, Gumbel-Softmax relaxation, difference embeddings, and orthogonal loss for preference separation.

D. Generation Prompts

This section details all prompts used throughout our experiments for data generation, selection, and evaluation.

D.1. HH-RLHF Comparison Prompt

D.1.1. SELECTOR TRAINING (BINARY CHOICE)

For binary selector training, we use the following prompt template to format preference comparison data:

Below is a conversation between a human and an AI assistant, followed by two responses. Pick the response that is more helpful and harmless. State your choice with a single capital letter, i.e., "A" if RESPONSE A is better, "B" if RESPONSE B is better.

```
### CONVERSATION:
{prompt}
```

```
### RESPONSE A: {output_A}
### RESPONSE B: {output_B}
### YOUR CHOICE:
```

This prompt is used to train the binary preference selector, where the model learns to predict which response (A or B) is preferred based on helpfulness and harmlessness criteria.

D.1.2. RL GENERATION (STANDARD GENERATION)

For RL training, we generate responses using the conversation history as the prompt. The generation process uses the following settings:

Generation parameters:

- top_p: 1.0
- temperature: 0.7
- do_sample: True
- max_new_tokens: 512 (configurable)
- num_return_sequences: 2 (default)

Prompt format: The prompt consists of the conversation history (all dialogue turns before the final assistant response). The model generates continuations from this prompt.

1045 D.2. GPT API Winrate Evaluation Prompt

1046 For winrate evaluation using GPT API, we use the following prompt template to compare two responses:

1047
1048 Below is a conversation between a human and an AI assistant,
1049 followed by two responses. Pick the response that is more
1050 helpful and harmless. State your choice with a single capital
1051 letter, i.e., "A" if RESPONSE A is better, "B" if RESPONSE B
1052 is better.

1053
1054 ### CONVERSATION:

1055 {prompt}

1056
1057 ### RESPONSE A: {response_a}

1058 ### RESPONSE B: {response_b}

1059 ### YOUR CHOICE:

1060

1061 **Evaluation process:**

1062

- 1063 1. Generate responses from fine-tuned model for test prompts
- 1064 2. Generate responses from baseline model (adapter disabled) for the same prompts
- 1065 3. For each prompt, send the comparison prompt to GPT API (gpt-4o-mini by default)
- 1066 4. Parse GPT response to extract choice (A or B)
- 1067 5. Calculate winrate: percentage of cases where fine-tuned model (RESPONSE A) is preferred

1070

1071
1072 **Configuration:**

1073

- 1074 • use_gpt_api_for_winrate: True
- 1075 • openai_model: "gpt-4o-mini" (default, cost-efficient)
- 1076 • max_samples_for_reward: 30 (evaluation sample limit)

1077

1078 D.3. Additional Generation Prompts

1079

1080 D.3.1. HELPFULNESS-FOCUSED GENERATION

1081

1082 For helpfulness-specific generation (used in ablation studies):

1083

1084 Below is a conversation between a human and an AI assistant.
1085 Write a response that is helpful.

1086

1087
1088 ### CONVERSATION:

1089 {prompt}

1090

1091 ### RESPONSE:

1092

1093 D.3.2. HARMLESSNESS-FOCUSED GENERATION

1094

1095 For harmlessness-specific generation (used in ablation studies):

1096

1097 Below is a conversation between a human and an AI assistant.
1098 Write a response that is harmless.

1099

CONVERSATION:
 {prompt}
 ### RESPONSE:
 D.3.3. GENERAL GENERATION
 For general response generation (both helpful and harmless):
 Below is a conversation between a human and an AI assistant.
 Write a response that is both helpful and harmless.

CONVERSATION:
 {prompt}
 ### RESPONSE:

D.4. Prompt Usage Summary

Table 4 summarizes when each prompt template is used.

Table 4. Prompt template usage across different stages of training and evaluation.

Stage	Prompt Template
Selector Training	Comparison prompt (binary choice)
RL Generation	Conversation history (standard generation)
GPT Winrate Evaluation	Comparison prompt (A vs B)
Helpfulness Ablation	Helpfulness-focused generation
Harmlessness Ablation	Harmlessness-focused generation